

Program on matrix using CUDA

Name: Satyam Singh

Reg no: 230905256

CSE B2 - 37

Lab Exercises:

Q1)

Write a program in CUDA to perform parallel Sparse Matrix-Vector multiplication using

compressed sparse row (CSR) storage format. Represent the sparse matrix in CSR format in host

code

Ans)

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <cuda_runtime.h>
```

```
__global__ void csrSpmvKernel(int numRows, int *rowPtr, int *colIdx,
```

```
int *values, int *x, int *y) {
```

```
int row = blockIdx.x * blockDim.x + threadIdx.x;
```

```
if (row < numRows) {
```

```
int sum = 0;
for (int j = rowPtr[row]; j < rowPtr[row + 1]; j++) {
    sum += values[j] * x[colldx[j]];
}
y[row] = sum;
}
}
```

```
int main() {
```

```
int N, nnz;
```

```
printf("Enter matrix size N: ");
```

```
scanf("%d", &N);
```

```
printf("Enter number of non-zero elements: ");
```

```
scanf("%d", &nnz);
```

```
int *rowInd = (int*)malloc(nnz * sizeof(int));
```

```
int *colInd = (int*)malloc(nnz * sizeof(int));
```

```
int *val = (int*)malloc(nnz * sizeof(int));
```

```
printf("Enter row col value (0-based indexing):\n");  
for (int i = 0; i < nnz; i++) {  
    scanf("%d %d %d", &rowInd[i], &colInd[i], &val[i]);  
}
```

```
int *rowPtr = (int*)calloc(N + 1, sizeof(int));  
int *colIdx = (int*)malloc(nnz * sizeof(int));  
int *values = (int*)malloc(nnz * sizeof(int));
```

```
// Build rowPtr
```

```
for (int i = 0; i < nnz; i++)  
    rowPtr[rowInd[i] + 1]++;
```

```
for (int i = 0; i < N; i++)  
    rowPtr[i + 1] += rowPtr[i];
```

```
int *temp = (int*)malloc(N * sizeof(int));  
for (int i = 0; i < N; i++)  
    temp[i] = rowPtr[i];
```

```
// Fill colIdx & values
```

```

for (int i = 0; i < nnz; i++) {
    int r = rowInd[i];
    int pos = temp[r];
    colldx[pos] = colInd[i];
    values[pos] = val[i];
    temp[r]++;
}

free(temp);

int *x = (int*)malloc(N * sizeof(int));
printf("Enter vector x:\n");
for (int i = 0; i < N; i++)
    scanf("%d", &x[i]);

int *y = (int*)calloc(N, sizeof(int));

// Device memory
int *d_rowPtr, *d_colldx, *d_values, *d_x, *d_y;

cudaMalloc(&d_rowPtr, (N + 1) * sizeof(int));

```

```

cudaMalloc(&d_colldx, nnz * sizeof(int));
cudaMalloc(&d_values, nnz * sizeof(int));
cudaMalloc(&d_x, N * sizeof(int));
cudaMalloc(&d_y, N * sizeof(int));

cudaMemcpy(d_rowPtr, rowPtr, (N + 1) * sizeof(int),
cudaMemcpyHostToDevice);
cudaMemcpy(d_colldx, colldx, nnz * sizeof(int),
cudaMemcpyHostToDevice);
cudaMemcpy(d_values, values, nnz * sizeof(int),
cudaMemcpyHostToDevice);
cudaMemcpy(d_x, x, N * sizeof(int), cudaMemcpyHostToDevice);

csrSpmvKernel<<<(N+255)/256, 256>>>(N, d_rowPtr, d_colldx, d_values,
d_x, d_y);

cudaMemcpy(y, d_y, N * sizeof(int), cudaMemcpyDeviceToHost);

printf("\nResult vector y:\n");
for (int i = 0; i < N; i++)
printf("%d\n", y[i]);
free(rowInd); free(colInd); free(val);

```

```
free(rowPtr); free(colIdx); free(values);

free(x); free(y);

cudaFree(d_rowPtr); cudaFree(d_colIdx);

cudaFree(d_values); cudaFree(d_x); cudaFree(d_y);

return 0;

}
```

Output:

```
• STUDENT@MIT-ICT-LAB5-03:~/Desktop/230905256/Lab9$ nvcc q1.cu -o q1
• STUDENT@MIT-ICT-LAB5-03:~/Desktop/230905256/Lab9$ ./q1
Enter matrix size N: 3
Enter number of non-zero elements: 4
Enter row col value (0-based indexing):
0 0 2
0 2 3
1 1 4
2 0 5
Enter vector x:
1 2 3

Result vector y:
11
8
5
```

Q2)

Write a program in CUDA to read MxN matrix A and replace 1st row of this matrix by same

elements, 2nd row elements by square of each element, 3rd row elements by cube of each element

and so on.

Ans)

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <math.h>
```

```
#include <cuda_runtime.h>
```

```
// Kernel
```

```
__global__ void transformMatrix(int *A, int M, int N) {
```

```
int col = blockIdx.x * blockDim.x + threadIdx.x;
```

```
int row = blockIdx.y * blockDim.y + threadIdx.y;
```

```
if (row < M && col < N) {
```

```
int idx = row * N + col;
```

```
A[idx] = pow(A[idx], row + 1);
```

```
}
```

```
}
```

```
int main() {
```

```
int M, N;
```

```
printf("Enter M and N: ");
```

```
scanf("%d %d", &M, &N);
```

```
int *A = (int*)malloc(M * N * sizeof(int));
```

```
printf("Enter matrix elements:\n");
```

```
for (int i = 0; i < M*N; i++)
```

```
scanf("%d", &A[i]);
```

```
int *d_A;
```



```
cudaMalloc(&d_A, M*N*sizeof(int));  
cudaMemcpy(d_A, A, M*N*sizeof(int), cudaMemcpyHostToDevice);  
  
dim3 threads(16,16);  
dim3 blocks((N+15)/16, (M+15)/16);  
  
transformMatrix<<<blocks, threads>>>(d_A, M, N);  
  
cudaMemcpy(A, d_A, M*N*sizeof(int), cudaMemcpyDeviceToHost);  
  
printf("\nTransformed Matrix:\n");  
for (int i = 0; i < M; i++) {  
    for (int j = 0; j < N; j++)  
        printf("%d ", A[i*N+j]);  
    printf("\n");  
}  
  
cudaFree(d_A);  
free(A);  
  
return 0;
```

}

Output:

```
STUDENT@MIT-ICT-LAB5-03:~/Desktop/230905256/Lab9$ nvcc q2.cu -o q2
STUDENT@MIT-ICT-LAB5-03:~/Desktop/230905256/Lab9$ ./q2
Enter M and N: 3 3
Enter matrix elements:
1 2 3
2 2 2
3 3 3

Transformed Matrix:
1 2 2
4 4 4
27 27 27
```

Q3) Write a CUDA program that reads a matrix A of size MxN and produce an output matrix B of same size such that it replaces all non-border elements of A with its equivalent 1's complement and remaining elements same as matrix A.

Ans)

```
#include <stdio.h>

#include <stdlib.h>

#include <cuda_runtime.h>

__device__ unsigned int get_mask(int n) {
if (n == 0) return 1;
```

```

unsigned int mask = n;

mask |= mask >> 1;

mask |= mask >> 2;

mask |= mask >> 4;

mask |= mask >> 8;

mask |= mask >> 16;


return mask;

}

__global__ void complementKernel(int *A, int *B, int M, int N) {

int row = blockIdx.y * blockDim.y + threadIdx.y;

int col = blockIdx.x * blockDim.x + threadIdx.x;


if (row < M && col < N) {

int idx = row * N + col;


if (row == 0 || row == M-1 || col == 0 || col == N-1) {

B[idx] = A[idx];

} else {

int val = A[idx];

```

```
if (val == 0)
B[idx] = 1;
else {
unsigned int mask = get_mask(val);
B[idx] = val ^ mask;
}
}
}
}
```

```
void printBinary(int n) {
int started = 0;

for (int i = 31; i >= 0; i--) {
int bit = (n >> i) & 1;
if (bit) started = 1;
if (started) printf("%d", bit);
}
if (!started) printf("0");
}
```

```
int main() {  
    int M, N;  
  
    printf("Enter M and N: ");  
    scanf("%d %d", &M, &N);  
  
    int *A = (int*)malloc(M*N*sizeof(int));  
    int *B = (int*)malloc(M*N*sizeof(int));  
  
    printf("Enter matrix elements:\n");  
    for (int i = 0; i < M*N; i++)  
        scanf("%d", &A[i]);  
  
    int *d_A, *d_B;  
    cudaMalloc(&d_A, M*N*sizeof(int));  
    cudaMalloc(&d_B, M*N*sizeof(int));  
  
    cudaMemcpy(d_A, A, M*N*sizeof(int), cudaMemcpyHostToDevice);  
  
    dim3 threads(16,16);
```

```

dim3 blocks((N+15)/16, (M+15)/16);

complementKernel<<<blocks, threads>>>(d_A, d_B, M, N);

cudaMemcpy(B, d_B, M*N*sizeof(int), cudaMemcpyDeviceToHost);

printf("\nResult Matrix:\n");
for (int i = 0; i < M; i++) {
    for (int j = 0; j < N; j++) {
        int val = B[i*N+j];

        if (i==0 || i==M-1 || j==0 || j==N-1)
            printf("%d ", val);
        else {
            printf("");
            printBinary(val);
            printf(" ");
        }
    }
    printf("\n");
}

```

```
cudaFree(d_A);
```

```
cudaFree(d_B);
```

```
free(A);
```

```
free(B);
```

```
return 0;
```

```
}
```

OUTPUT:

```
1 error detected in the compilation of "q3.cu".
• STUDENT@MIT-ICT-LAB5-03:~/Desktop/230905256/Lab9$ nvcc q3.cu -o q3
• STUDENT@MIT-ICT-LAB5-03:~/Desktop/230905256/Lab9$ ./q3
Enter M and N: 3 3
Enter matrix elements:
1 2 3
4 5 6
7 8 9

Result Matrix:
1 2 3
4 '10' 6
7 8 9
```